

Implementing Shor: Quantum Period Finding

24, 26 November · Objective 7 Thursday 26 November: Thanksgiving Break — no class

Building the pieces: the quantum Fourier transform, modular exponentiation, and the continued-fraction post-processing that turns a measured integer into a period. Then an honest accounting of why nobody has factored anything interesting.

The quantum Fourier transform

The QFT maps computational basis states to Fourier basis states:

$$\text{QFT } |x\rangle = (1/\sqrt{Q}) \sum_{y=0}^{Q-1} e^{2\pi i xy/Q} |y\rangle \quad Q = 2^n$$

The circuit is Hadamards interleaved with controlled phase rotations:

```
import numpy as np
from qiskit import QuantumCircuit

def qft(n):
    qc = QuantumCircuit(n, name="QFT")
    for j in range(n):
        qc.h(j)
        for k in range(j + 1, n):
            # Controlled rotation by  $\pi/2^{k-j}$ : encodes the pairwise phase
            # relationship that makes the transform separable.
            qc.cp(np.pi / 2**(k - j), k, j)
    for j in range(n // 2):
        qc.swap(j, n - 1 - j)      # QFT reverses bit order
    return qc
```

$O(n^2)$ gates for a transform over 2^n amplitudes — against the classical FFT's $O(Q \log Q)$. As Week 14 established, this is not usable for signal processing, because the output is sampled rather than read. It is usable when one frequency dominates.

Approximate QFT. Rotations by $\pi/2^k$ with large k are below hardware precision and can be dropped, reducing to $O(n \log n)$ gates with negligible error. This is standard in resource estimates.

Modular exponentiation

The expensive part — and the reason Shor is hard to run. You need a reversible circuit for:

$$|x\rangle|1\rangle \rightarrow |x\rangle|a^x \bmod N\rangle$$

built from repeated controlled modular multiplication, each from controlled modular addition.

Resource requirements for an n -bit N :

```
Qubits:           ~2n + 3           (2n counting register + n work + ancillas)
Toffoli gates: ~O(n³)
For RSA-2048: ~4100 logical qubits, ~10⁹ Toffoli gates
```

Modular exponentiation dominates the circuit — the QFT is a rounding error beside it. This is why "how many qubits to break RSA" has an answer in the thousands of *logical* qubits and the millions of *physical* ones.

Continued fractions

Measurement returns $y \approx (Q/r) \cdot k$ for some integer k , so $y/Q \approx k/r$. Recovering r from that approximation is a classical number-theory step:

```
from fractions import Fraction

def period_from_measurement(y, Q, N, a):
    frac = Fraction(y, Q).limit_denominator(N)    # continued-fraction convergent
    r = frac.denominator
    if r % 2 == 0 and pow(a, r, N) == 1:
        return r
    return None
```

`limit_denominator` implements the continued-fraction expansion, returning the best rational approximation with denominator below N . It succeeds with constant probability per run, so a few repetitions suffice.

Full implementation, $N = 15$

```
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
from qiskit.circuit.library import QFT
import numpy as np
from math import gcd
from fractions import Fraction

def amod15(a, power):
    """Controlled  $a^{(2^{\text{power}})} \bmod 15$  – hardcoded permutations.
    Real implementations build this from modular adders; these shortcuts
    exist because the general circuit is too deep to simulate."""
    if a not in [2, 4, 7, 8, 11, 13]:
        raise ValueError("a must be coprime to 15")
    U = QuantumCircuit(4)
    for _ in range(power):
        if a in [2, 13]: U.swap(2,3); U.swap(1,2); U.swap(0,1)
        if a in [7, 8]: U.swap(0,1); U.swap(1,2); U.swap(2,3)
        if a in [4, 11]: U.swap(1,3); U.swap(0,2)
        if a in [7, 11, 13]:
            for q in range(4): U.x(q)
    U = U.to_gate(); U.name = f"{a}^{2**power} mod 15"
    return U.control()

def shor_circuit(a, n_count=8):
    qc = QuantumCircuit(n_count + 4, n_count)
    qc.h(range(n_count)) # superposition over all exponents
    qc.x(n_count) # work register -> |1>
    for q in range(n_count):
        qc.append(amod15(a, q), [q] + list(range(n_count, n_count + 4)))
    qc.append(QFT(n_count, inverse=True), range(n_count)) # frequency -> position
    qc.measure(range(n_count), range(n_count))
    return qc

a, N, n_count = 7, 15, 8
qc = shor_circuit(a, n_count)
sim = AerSimulator()
counts = sim.run(transpile(qc, sim), shots=2048).result().get_counts()

for y_str, c in sorted(counts.items(), key=lambda kv: -kv[1]):
    y = int(y_str, 2)
    r = Fraction(y, 2**n_count).limit_denominator(N).denominator
    print(f"y={y:>3} y/Q={y/2**n_count:.3f} r={r}", end="")
    if r % 2 == 0 and pow(a, r, N) == 1:
        f = [gcd(pow(a, r//2, N) + s, N) for s in (-1, 1)]
        print(f" -> factors {f}")
    else:
        print(" -> retry")
```

```

y= 0  y/Q=0.000  r=1  -> retry
y= 64  y/Q=0.250  r=4  -> factors [3, 5]
y=128  y/Q=0.500  r=2  -> retry
y=192  y/Q=0.750  r=4  -> factors [3, 5]

```

Peaks at multiples of $Q/r = 256/4 = 64$. Two of four measurements yield the period; the others are handled by rerunning. $15 = 3 \times 5$.

Analogy boundary — Shor as a factoring service. The `amod15` shortcuts above are hardcoded permutations for one modulus, not a general implementation, and this is the standard practice in demonstrations. Published "factorizations" of 15, 21, and 35 have almost all used circuits **compiled with knowledge of the answer** — Smolin, Smith, and Vargo showed in 2013 that such demonstrations can be reduced to near-trivial circuits. No general-purpose Shor implementation has factored a number that classical methods could not factor instantly. When you read a factoring headline, the question to ask is whether the modular exponentiation was general or compiled against the known factors.

Why RSA-2048 is not imminent

REQUIREMENT	ESTIMATE	CURRENT (2026)
Logical qubits	$\sim 4,100$	$\sim 10^1 - 10^2$
Physical qubits	$\sim 2 \times 10^7$	$\sim 10^3$
Toffoli gates	$\sim 10^9$	
Runtime	~ 8 hours	
Physical error rate	$< 10^{-3}$	$\sim 10^{-3}$ ← threshold reached

The encouraging line is the last: physical error rates have reached the fault-tolerance threshold. The gap is now **overhead** — roughly $10^3 - 10^4$ physical qubits per logical qubit — and that is an engineering problem rather than a physics one. Estimates for the arrival of cryptographically relevant machines cluster in the 2030s with wide error bars, and that uncertainty is precisely why harvest-now-decrypt-later makes migration a present-tense concern.

Key takeaways

- The QFT costs $O(n^2)$ gates, reducible to $O(n \log n)$ by dropping sub-precision rotations.
- Modular exponentiation dominates Shor's cost at $O(n^3)$ Toffolis — the QFT is negligible beside it.

- Continued fractions recover r from the measured $y/Q \approx k/r$; a few repetitions suffice.
- Published small-number factorizations largely use circuits compiled with prior knowledge of the factors.
- Physical error rates have reached threshold; the remaining barrier is error-correction overhead, measured in millions of physical qubits.

Self-check

Q1 (*Objective 7*) — Your Shor run on $N = 15$, $a = 7$ returns $y = 128$. Work through the post-processing and explain why this outcome is uninformative despite being a legitimate peak.

Q2 (*Objective 7*) — Explain why the counting register needs $\sim 2n$ qubits for an n -bit N , rather than n .

Check your answers

A1 — Post-processing $y = 128$ with $Q = 256$:

```
y/Q = 128/256 = 1/2
Fraction(1,2).limit_denominator(15) → 1/2 → r = 2
Check: 72 = 49 = 4 (mod 15) ≠ 1 → r = 2 is NOT the period → retry
```

The true period is 4, and the peaks sit at multiples of $Q/r = 64$: $y \in \{0, 64, 128, 192\}$. The value 128 is a genuine peak — but $128/256$ reduces to $1/2$, and the continued-fraction expansion returns the denominator of the **reduced** fraction. The factor of 2 shared by $k = 2$ and $r = 4$ cancels, and the information about r is destroyed by the reduction.

This is the general failure mode: measuring $y = k \cdot Q/r$ yields k/r in lowest terms, so whenever $\gcd(k, r) > 1$ you recover a proper divisor of r rather than r itself. Here $k = 2$, $\gcd(2, 4) = 2$, and you get $r/2 = 2$.

The remedy is built into the algorithm: **verify and retry**. Check $\text{pow}(a, r, N) == 1$; if it fails, rerun. Since the number of k values coprime to r is $\phi(r)$, the success probability per run is $\phi(r)/r$, which is $\Omega(1/\log \log r)$ — good enough that a handful of runs suffices. You can also test small multiples of the recovered candidate, which often finds r without another quantum run. Note that $y = 0$ is likewise a legitimate peak carrying no information, for the same reason.

A2 — The counting register must resolve the period, and the requirement comes from the resolution of the Fourier peak.

The measured y approximates $k \cdot Q/r$. To recover r by continued fractions you need y/Q close enough to k/r that k/r is the *unique* rational with denominator below N in that neighborhood. Two distinct fractions with denominators under N can differ by as little as:

$$|k_1/r_1 - k_2/r_2| = |k_1 r_2 - k_2 r_1| / (r_1 r_2) \geq 1/N^2$$

So you need y/Q accurate to better than $1/(2N^2)$. With $Q = 2^{\{n_count\}}$ and $N \approx 2^n$:

$$1/Q < 1/(2N^2) = 1/(2 \cdot 2^{\{2n\}}) \quad \Rightarrow \quad n_count \geq 2n + 1$$

Hence $\sim 2n$ counting qubits. With only n , the resolution is $\sim 1/N$ and multiple candidate fractions fall within the measurement's uncertainty — the continued-fraction step cannot distinguish them, and r is unrecoverable.

The intuition worth keeping: the counting register sets the **frequency resolution** of the QFT, exactly as record length sets resolution in a classical DFT. Periods up to N require resolving frequencies as fine as $1/N$, and distinguishing *nearby* such frequencies requires $1/N^2$ — so the register must be twice as wide as the number you are factoring. For RSA-2048 that is ~ 4096 counting qubits, which is where the ~ 4100 logical qubit figure originates.